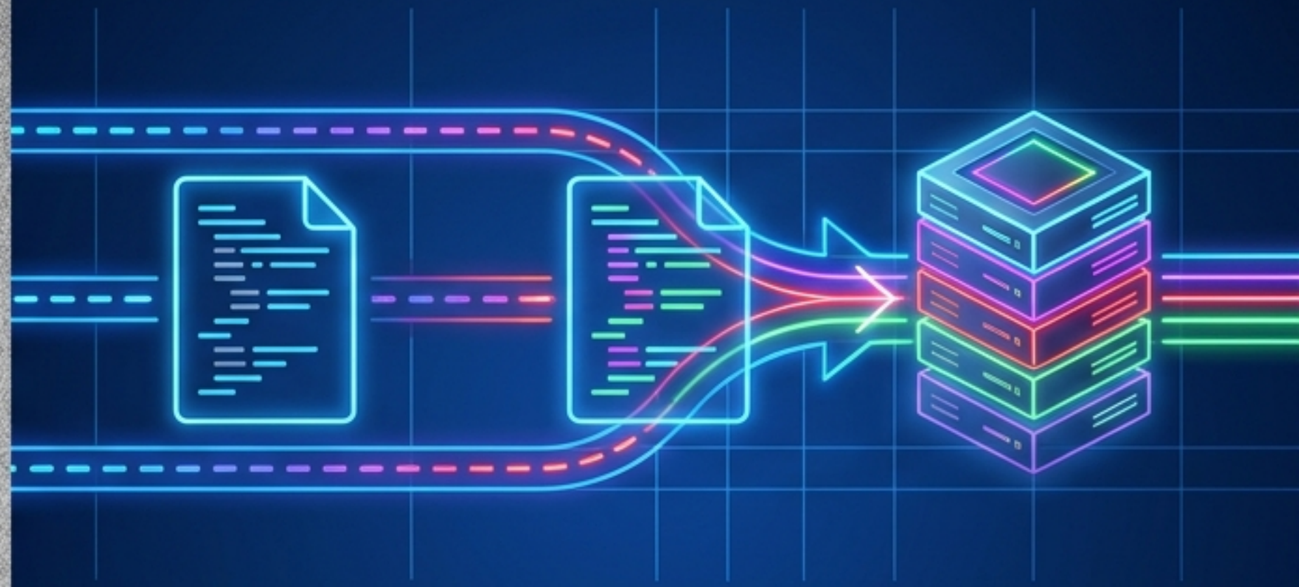
Infrastruktur, Konfigurering & Deployment
Att bygga det automatiserade IT-systemet (20 YH-poäng)
Vecka 44–47 | Från manuella klick till Infrastructure as Code

NotebookLM

Det manuella mörkret

- Individuella, isolerade skript
- Klicka manuellt i gränssnitt
- Hög felmarginal och personberoende

Den automatiserade fabriken



- Infrastruktur som kod (IaC)
- Automatisk mjukvaruinsta-llation
- Självvaliderande system

Insikt: Målet är inte att lära sig fyra isolerade språk, utan att orkestrera dem till en enda fungerande kedja.

NotebookLM

Den automatiserade kedjan i praktiken

PowerShell
[Motorn & Verifieraren]

Terraform
[Grunden (IaC)]

Ansible
[Konfiguratören]

VG-Nivå

Python
[Dirigenten]

VG-Nivå

VG-Nivå

PowerShell
[Motorn & Verifieraren]

Hantera systemadministration, filhantering och utför lokala hälsokontroller.

Terraform
[Grunden (IaC)]

Provisionerar resurser, bygger katalogstrukturer och genererar konfigurationsfiler.

Ansible
[Konfiguratören]

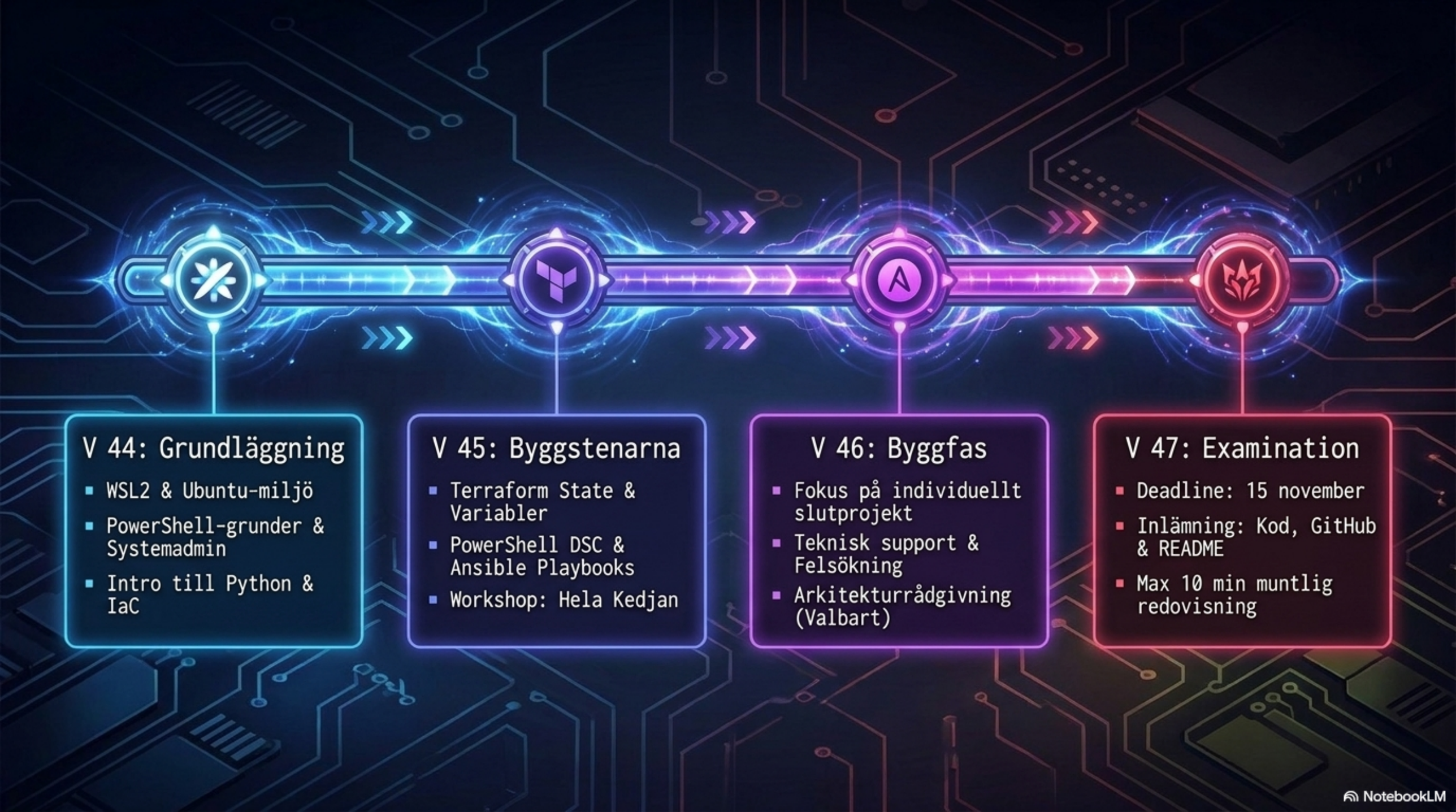
Agentlös hantering av mjukvara och systeminställningar via deklarativa YAML-playbooks.

Python
[Dirigenten]

Orkestrerar hela kedjan, utför externa HTTP-verifieringar och bygger slutrapporter.

Insikt: Målet är inte att lära sig fyra isolerade språk, utan att orkestrera dem till en enda fungerande kedja.

NotebookLM


V 44: Grundläggning
WSL2 & Ubuntu-miljö
PowerShell-grunder & Systemadmin
Intro till Python & IaC
V 45: Byggstenarna
Terraform State & Variabler
PowerShell DSC & Ansible Playbooks
Workshop: Hela Kedjan
V 46: Byggfas
Fokus på individuellt slutprojekt
Teknisk support & Felsökning
Arkitekturrådgivning (Valbart)
V 47: Examination
Deadline: 15 november
Inlämning: Kod, GitHub & README
Max 10 min muntlig redovisning
NotebookLM

Kärnkonceptet: Imperativ vs. Deklarativ Metodik

Imperativt
(PowerShell / Bash)

Fokus: HUR gör vi det?

Steg-för-steg instruktioner.
Ordningen på koden är absolut kritisk.

“Skapa mappen, kopiera sedan filen, och starta till sist tjänsten.”

Svårare att återställa vid en krasch.
Kräver manuell felhantering.

Deklarativt
(Terraform / Ansible)

Fokus: VAD är målet?

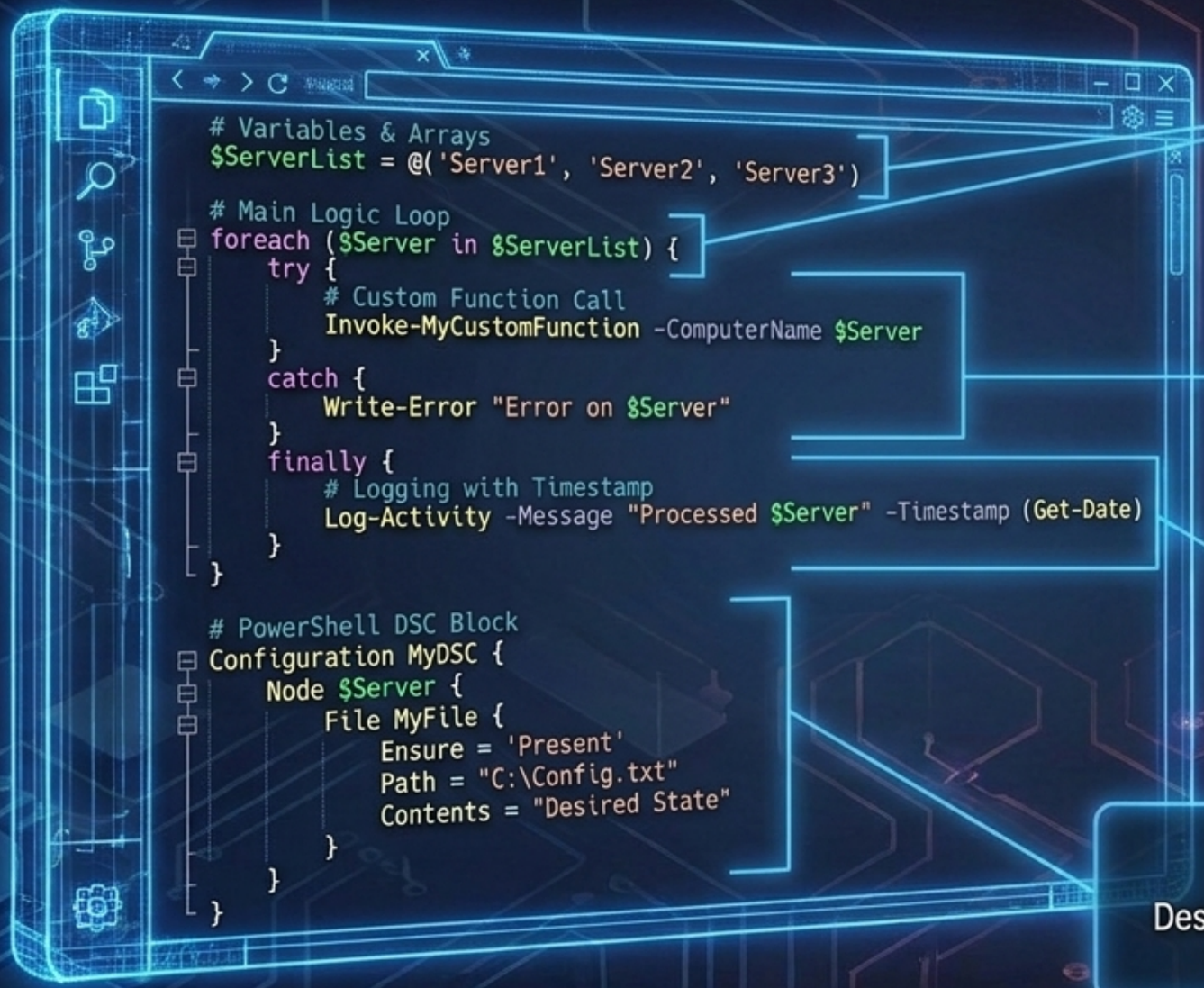
Beskriver ett önskat slutläge.
Ordningen i koden saknar betydelse.

“Mappen ska finnas, filen ska ligga där, och tjänsten ska vara igång.”

Idempotent. Enkelt att applicera koden på nytt utan att förstöra systemet.
Verktyget minns tillståndet (State).

NotebookLM

Dykning 1: PowerShell (Motorn)



```powershell
# Variables & Arrays
$ServerList = @('Server1', 'Server2', 'Server3')

# Main Logic Loop
foreach ($Server in $ServerList) {
    try {
        # Custom Function Call
        Invoke-MyCustomFunction -ComputerName $Server
    }
    catch {
        Write-Error "Error on $Server"
    }
    finally {
        # Logging with Timestamp
        Log-Activity -Message "Processed $Server" -Timestamp (Get-Date)
    }
}

# PowerShell DSC Block
Configuration MyDSC {
    Node $Server {
        File MyFile {
            Ensure = 'Present'
            Path = "C:\Config.txt"
            Contents = "Desired State"
        }
    }
}
```

Logik & Struktur

Variabler, listor och foreach-loopar hanterar data effektivt. Minst 3 egna funktioner krävs.

Felhantering

Try/Catch-block är obligatoriska. Förhindrar okontrollerade systemkraschar.

Loggning (VG)

Spårbarhet med detaljerad loggning, inklusive dynamiska tidsstämplar (Get-Date).

PowerShell DSC

Desired State Configuration. PowerShells inbyggda deklarativa städmaskin (Ensure = Present).

NotebookLM

Dykning 2: Infrastructure as Code (Terraform)

Fokuspunkt: Vi utnyttjar Terraforms variabler (Variables) och resultat (Outputs) för att generera hela projektets gemensamma konfigurationsfil (JSON).

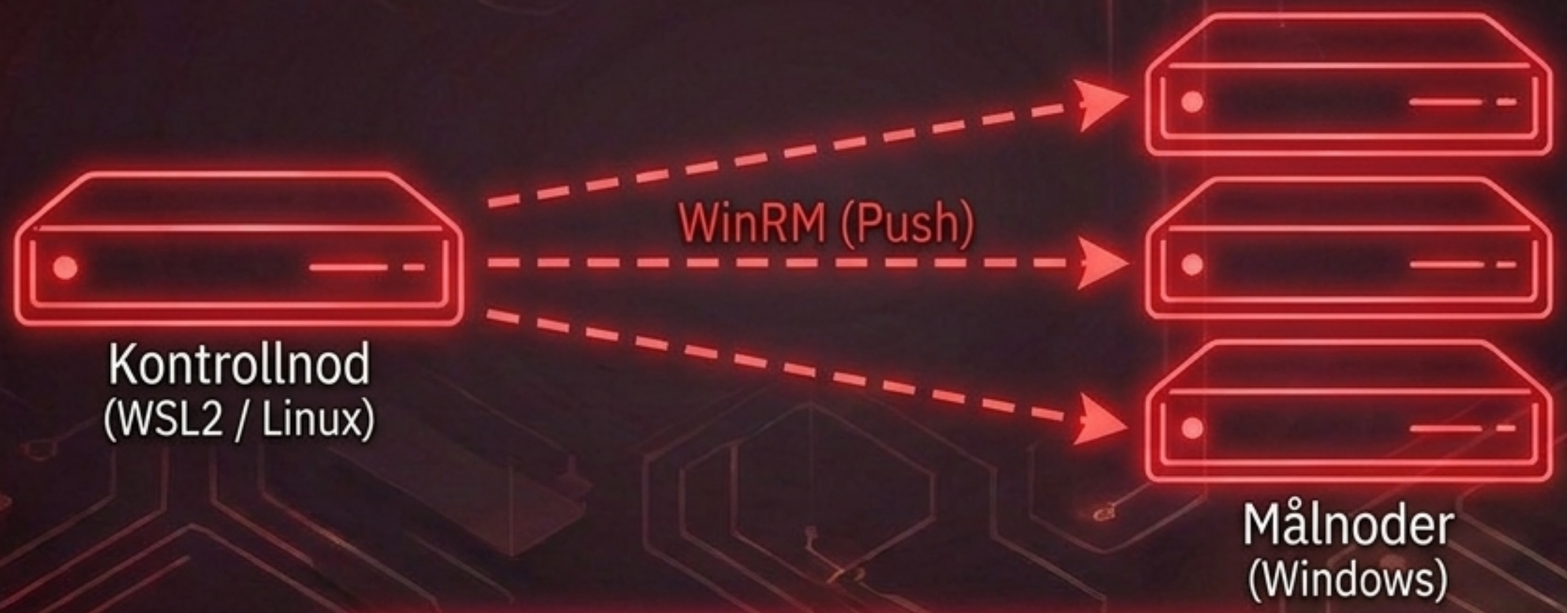
NotebookLM

Dykning 3: Konfigurationshantering (Ansible)

VG-Krav

Agentlöst
Kräver ingen förinstallerad mjukvara på målnoderna. Allt styrs utifrån.

Inventory & Playbooks
Inventory listar maskinparken. YAML-Playbooks agerar deklarativt recept för t.ex. IIS eller Nginx.



Smarta Handlers
Tjänster startas endast om en konfiguration faktiskt har ändrats.

NotebookLM

Dykning 4: Orkestrering (Python)

VG-Krav

Limmet (subprocess)

Kör hela automationskedjan automatisk, steg för steg, från ett enda startkommando.

Oberoende Verifiering

Använder ‘requests’ för att bevisa att systemet (t.ex. webbservern) faktiskt svarar med HTTP 200.

Styr Terraform

Styr Ansible

Python

(Spindeln i nätet)

Styr PowerShell

Central Loggning

Skapar en sömlös, sammanhängande logg över hela automationskedjans förlopp.

NotebookLM

SYNTES: Hela Kedjan (The Data Flow)

VG-Krav

Python Orkestrator (Kör allt, loggar och utför slutlig HTTP-test)

Station 1:
Provisionerar struktur & genererar config.json (Namn, Port, Miljö).

Station 2:
Läser config.json. Konfigurerar systemet helt baserat på filens värden.

Station 3:
Gör lokal systemverifiering på Windows och skriver tidsstämplad logg.

Station 4:
Utför slutgiltig, oberoende verifiering mot systemets publika endpoint.

Takeaway: JSON-filen är det bindande kontraktet som gör att verktygen kan prata med varandra.

NotebookLM

Slutprojektet (Examinationen)

MISSION BRIEFING: AUTOMATION

Huvuduppdrag

Automatisera installation och konfiguration av en valfri IT-tjänst från noll till drift.

Obligatoriska Filer

- .ps1 (PowerShell-skript)
- .tf (Terraform-infrastruktur)
- README.md (Dokumentation)

VG-Krav

- .py (Python) & .yml (Ansible)

Leverans

- Deadline: Senast 15 november
- Format: .zip-fil på plattformen + öppen länk till GitHub-repo

Slutprovet

Max 10 minuter muntlig genomgång. Fokus ligger på att försvara designval och bevisa total kodförståelse.

NotebookLM

Betygsmatris: G vs. VG

Kravnivå: Godkänt (G)

Fungerande Automation

- Kedja: Terraform -> PowerShell
- PowerShell: Minst 3 funktioner, loopar, villkor
- Stabilitet: Enkel felhantering (Try/Catch)
- Versionshantering: Minst 5 Git-commits
- Dokumentation: README med 5 grundavsnitt

Kravnivå: Väl Godkänt (VG)

Den Komplexa Kedjan

- Kedja: Terraform -> Ansible -> PowerShell -> Python
- Optimering: Djup parametrisering & specifika felmeddelanden
- Spårbarhet: Detaljerad, tidsstämplad loggning
- Versionshantering: Minst 10+ logiska Git-commits
- Dokumentation: 7 avsnitt i README, inkl. installationsguide

NotebookLM

Obligatoriska Spelregler

1. Dokumentation (README.md)

Inte bara en textfil. Måste innehålla Mål, Metod, Resultat, Reflektion och en klickbar GitHub-länk. Reflektionen är kritisk.

2. Versionshantering (Git)

Gör små commits ofta. ”Allt på en gång”-commits accepteras inte. Varje färdigt byggsteg ska motsvara en commit.

3. AI-Policyn (Kritisk Varning)

AI är tillåtet för felsökning och stöd.
MEN: Du måste förstå 100% av koden du lämnar in.
Om du under redovisningen inte kan förklara eller modifiera din egen kod resulterar det omedelbart i IG (Underkänt).

NotebookLM

Projekt-Blueprints (Inspiration)

Klassikern

Webbserver-automation

Terraform skapar struktur -> Ansible/PS installerar IIS/Nginx -> Python utför externt HTTP-test.

Administratören

Användarhantering

Terraform genererar JSON med personaldata -> PowerShell loopar och skapar konton/rättigheter -> Python loggar resultat.

Säkerhetsvakten

Backup & Övervakning

Terraform säkrar lagring -> PowerShell övervakar tjänster, utför datakopiering och larmar vid avvikelser.

”Enkel kod som fungerar – och som du själv förstår – är alltid bättre än avancerad kod du inte kan förklara.”

NotebookLM